

Saima mehwesh syeda

DRAW IT OR LOSE IT

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	4/10/26	SAIMA MEHWESH	Completed the software design document for Project Two, including the platform evaluation for server-side hosting, client-side compatibility, and development tools.

Executive Summary

The Gaming Room currently has an Android version of Draw It or Lose It and would like to expand the game so that it can be accessed from multiple platforms. The goal is to provide players with a modern web-based application that can be used on desktop operating systems such as Windows, Mac, and Linux, as well as mobile devices such as Android and iOS. Because the application will be web-based, the client will need a reliable server environment that can support many users while also providing compatibility with different browsers and devices.

The proposed solution is to use a web-based architecture in which the main application and game data are hosted on a server and users access the application through a web browser. The client side can use HTML, CSS, and JavaScript to create a responsive interface that adjusts to different screen sizes. This approach reduces the need to create a completely separate application for every supported operating system. The server can use a platform such as Linux, Windows Server, or another supported server environment to host the web application and manage communication between clients.

The Gaming Room should consider the cost, scalability, security, browser compatibility, and development expertise required for each platform. Linux is a strong option for the server because it is widely used for web hosting, has many available server tools, and can be deployed without the same operating-system licensing costs associated with Windows Server. For the client side, a responsive web application provides a practical way to support desktop and mobile users while reducing development and maintenance requirements.

Requirements

The Gaming Room requires Draw It or Lose It to support multiple teams and players while allowing users to access the game from multiple platforms. A game must be able to contain one or more teams, and each team must support multiple players. Game and team names must be unique so that users can determine whether a name is already being used. The application must also prevent multiple instances of the game service from existing in memory at the same time.

For Project Two, the application must be expanded into a web-based environment. The server must be capable of hosting the web application and supporting potentially thousands of players. The client application must provide a responsive HTML interface that works across desktop browsers and mobile devices. The development process must also consider programming languages, development tools, browser compatibility, development time, cost, and the expertise required to support the different platforms.

Design Constraints

The main design constraint is that Draw It or Lose It needs to operate in a distributed, web-based environment. Instead of depending on one specific operating system, the application should allow clients using Windows, Mac, Linux, Android, and iOS to communicate with the same back-end service. This requires the application to use common web standards so that users can access it through supported browsers.

Another constraint is scalability. The server must be capable of supporting a large number of players at the same time. The application should therefore be hosted on a server platform that can support web-server software, networking, databases, and additional server resources as the number of users increases. Cloud-based infrastructure could also be used to increase resources when necessary.
Browser and device compatibility are additional constraints. Different browsers and devices can display web content differently, so the development team must test the application using major browsers and different screen sizes. Responsive HTML and CSS should be used so that the interface works on desktop computers, tablets, and smartphones without requiring a completely separate interface for every device.
Cost is also a constraint. The Gaming Room will need to consider operating-system licensing, development software, hosting, testing devices, and developer expertise. Linux can reduce server operating-system licensing expenses, while Windows Server may require additional licensing costs. Development tools can also vary in cost depending on whether the team chooses free tools or commercial products.
Security and network reliability are important constraints because the game will communicate over the internet. The application should use secure connections such as HTTPS, protect user information, validate data received from clients, and properly handle network interruptions. Since the application depends on communication between clients and the server, the system should also be designed to handle temporary connectivity problems without causing the entire application to fail.

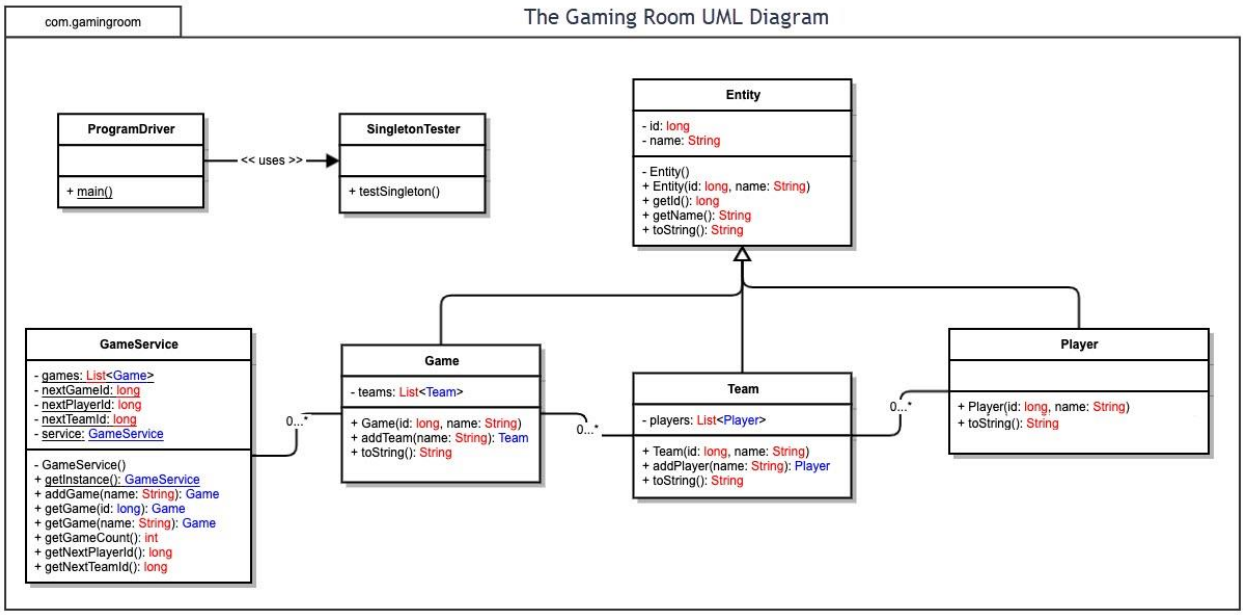
Domain Model

The UML class diagram represents the main objects used by Draw It or Lose It and shows how the classes work together. The Entity class acts as a **base class for** the other game objects. It contains the common id and name attributes, along with methods such as getId(), getName(), and toString(). This allows the common information and behavior to be defined in one location instead of being duplicated in the Game, Team, and Player classes.

The Game, Team, and Player classes inherit from the Entity class. This demonstrates inheritance because each of these classes can use the attributes and methods provided by the base class. The Game class contains a list of teams, while each Team contains a list of players. This creates a hierarchical relationship in which a game can have multiple teams and each team can have multiple players. The diagram represents these relationships using associations between the classes.

The GameService class manages the games and provides methods for adding and finding games. It also contains identifiers for games, teams, and players. The GameService class uses the Singleton pattern so that only one instance of the service exists in memory. This helps prevent multiple game services from independently managing the same game information.

The design also demonstrates encapsulation because the data and behaviors of each object are organized inside their respective classes. The use of inheritance through the Entity class reduces duplicated code, while polymorphism can be supported through methods such as toString() that are implemented by different classes. Overall, the domain model provides a structured way to represent games, teams, and players while supporting the requirements for unique identifiers and organized relationships between objects.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	**Mac**	**Linux**	**Windows**	**Mobile Devices**

Server Side	macOS can host web servers such as Apache and supports common server technologies. However, it is mainly a desktop operating system and is less suitable for a production server serving thousands of players. Apple hardware can also increase costs.	Linux is well suited for web hosting and supports servers such as Apache and Nginx. It is scalable, reliable, and many distributions have no operating-system licensing cost. The main weakness is that Linux administration may require additional technical expertise.	Windows Server can host web applications and provides user-friendly administration tools. It is reliable and widely supported, but Windows Server licensing can increase costs. It is a good option if the development team already has Windows expertise.	**Android and iOS are designed primarily as client platforms rather than** production servers. They should connect to a dedicated web server instead of hosting the application. This avoids limitations involving battery life, resources, and mobile connectivity.
Client Side	Users can access the application through Safari, Chrome, or Firefox. A responsive HTML interface reduces the need for a separate Mac application. Testing on Safari is important to ensure browser compatibility.	Linux users can access the game through supported browsers such as Chrome or Firefox. Standard HTML, CSS, and JavaScript can be used across Linux distributions, reducing development costs. Browser and distribution differences should still be tested.	Windows users can access the game through Edge, Chrome, or Firefox. Responsive web design allows the same client application to work across Windows devices. Testing different browsers and screen sizes will require additional development time.	The application should use responsive HTML, CSS, and JavaScript to support Android and iOS. Developers must test touch controls, screen sizes, and mobile browsers. This approach is less expensive than creating separate native applications.

Development Tools	Developers can use HTML, CSS, JavaScript, Java, Visual Studio Code, Eclipse, Git, and Xcode when needed. Most tools have free options, although some commercial tools may have licensing costs.	< Linux supports HTML, CSS, JavaScript, Java, Visual Studio Code, Eclipse, IntelliJ IDEA, and Git. Many tools are free, reducing development costs. Developers may need additional Linux and command-line expertise.	Windows supports HTML, CSS, JavaScript, Java, Visual Studio Code, Visual Studio, Eclipse, and Git. Many tools have free versions, while some Microsoft products require licenses. Windows expertise is widely available.	HTML, CSS, and JavaScript can support both Android and iOS through mobile browsers. Developers can use Visual Studio Code and browser developer tools. Native development would require Android Studio for Android and Xcode for iOS, increasing time and expertise requirements.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. Operating Platform: I recommend Linux, specifically Ubuntu Server, for hosting Draw It or Lose It. Linux is a good choice because it is reliable, supports web applications, and can help reduce operating costs. The server would manage the game while players access it through browsers on Windows, Mac, Android, and iOS devices. This means The Gaming Room would not have to build a completely separate application for every operating system. If more people start playing, the company can also increase its server resources.

2. Operating Systems Architectures: Linux manages computer resources through its kernel, which handles tasks such as running programs, managing memory, and handling network connections. Ubuntu Server can run the services needed for the game without requiring a full desktop interface. I recommend separating the application into three parts: the user interface, the game server, and the database. The user interface allows players to interact with the game, the server processes their actions, and the database saves important information. Windows, Mac, Android, and iOS work differently internally, but their web browsers can still communicate with the same Linux server. This makes it easier to support different devices without maintaining separate versions of the main game system.

3. Storage Management: I recommend using the ext4 file system and database such as MySQL. Ext4 would organize the server's files and folders, while MySQL would store information such as players, teams, games, and scores. A database is useful because the game needs to find and update information as players join teams and participate in games. Rules such as unique game and team names can help prevent duplicate records. The Gaming Room should also back up its database regularly and keep backup copies separate from the main server. This would make it easier to recover important information if something goes wrong. The company should monitor storage space as well so the server does not run out of room.

4. Memory Management: Linux uses virtual memory to manage how programs use RAM. Each process has its own virtual address space, which helps keep programs separate. Linux also uses paging to manage memory in smaller units. If RAM becomes limited, it can move some memory data to swap space on the disk, although this is slower than using RAM directly. These techniques help the server run multiple game sessions at once. If the application uses Java, the Java Virtual Machine can also use garbage collection to free memory that is no longer needed. The Gaming Room should monitor memory usage and make sure the server has enough RAM for its players. If the game becomes more popular, the company may need to add memory or use additional servers to keep the application running smoothly.

5. Distributed Systems and Networks: A client-server system would allow players using different devices to participate in the same game. Their browsers would send requests to the Linux server over the internet using HTTPS. The server would process those requests and send information back to the players. If the game needs frequent updates during a round, WebSockets could help send updates between the server and connected players. For example, when one player makes a move, the server checks the action, updates the game, and shares the

change with the other players. This helps everyone see the same game information. The system also depends on a working internet connection, an available server, and access to the database. If a player disconnects, the game should show a message and try to reconnect. If the server goes down, players may lose access to their games. Regular backups, server monitoring, and recovery plans can help reduce these problems. If the number of players increases, the company could use multiple servers and a load balancer to spread the workload.

6. Security: The Gaming Room needs to protect player information and prevent unauthorized access. The application should use HTTPS to protect information sent between players and the server. Passwords should never be saved as plain text. Instead, they should be stored using a secure password-hashing method. Ubuntu Server provides security features such as file permissions, firewalls, and software updates. The company should limit access to the server, remove unnecessary services, and install security updates regularly. Database access should also be restricted to authorized users and services. The application should check information received from players before processing it. Using parameterized database queries can help prevent SQL injection attacks. The company should also protect its backups and avoid saving passwords or authentication tokens in logs. The server must check whether a player is allowed to perform an action instead of automatically trusting information sent by the player's device. These steps would help protect user information and make the game safer across different platforms.